- `ActiveRecord::QueryCache` caches all SELECT queries generated in a request. If any INSERT or UPDATE takes place then the cache is cleaned.
- `ActionDispatch::Cookies` sets cookies for the request.
- `ActionDispatch::Session::CookieStore` is responsible for storing the session in cookies. An alternate middleware can be used for this by changing the `config.action_controller.session_store` to an alternate value. Additionally, options passed to this can be configured by using `config.action_controller.session_options`.
- `ActionDispatch::Flash` sets up the `flash` keys. Only available if `config.action_controller.session_store` is set to a value.
- `ActionDispatch::ParamsParser` parses out parameters from the request into `params`.
- `Rack::MethodOverride` allows the method to be overridden if `params[:_method]` is set. This is the middleware which supports the PATCH, PUT, and DELETE HTTP method types.
- `Rack::Head` converts HEAD requests to GET requests and serves them as so.

Besides these usual middleware, you can add your own by using the `config.middleware.use`method:

```
config.middleware.use Magical::Unicorns
```

This will put the `Magical::Unicorns` middleware on the end of the stack. You can use `insert_before` if you wish to add a middleware before another.

```
config.middleware.insert_before Rack::Head, Magical::Unicorns
```

There's also `insert_after` which will insert a middleware after another:

```
config.middleware.insert_after Rack::Head, Magical::Unicorns
```

Middlewares can also be completely swapped out and replaced with others:

```
config.middleware.swap ActionController::Failsafe, Lifo::Failsafe
```

They can also be removed from the stack completely:

```
config.middleware.delete "Rack::MethodOverride"
```

3.5 Configuring i18n

All these configuration options are delegated to the `I18n` library.

- `config.i18n.available_locales` whitelists the available locales for the app. Defaults to all locale keys found in locale files, usually only `:en` on a new application.
- `config.i18n.default_locale` sets the default locale of an application used for i18n. Defaults to `:en`.
- `config.i18n.enforce_available_locales` ensures that all locales passed through i18n must be declared in the `available_locales` list, raising an `I18n::InvalidLocale`exception when setting an unavailable locale. Defaults to `true`. It is recommended not to disable this option unless strongly required, since this works as a security measure against setting any invalid locale from user input.
- `config.i18n.load_path` sets the path Rails uses to look for locale files. Defaults to `config/locales/*.{yml,rb}`.

3.6 Configuring Active Record

`config.active_record` includes a variety of configuration options:

- `config.active_record.logger` accepts a logger conforming to the interface of Log4r or the default Ruby Logger class, which is then passed on to any new database connections made. You can retrieve this logger by calling `logger` on either an Active Record model class or an Active Record model instance. Set to `nil` to disable logging.